Proyecto Final Momento II

Diseño y Análisis: Diagrama de Clases (Capa Lógica)

TEJO CÓSMICO

Programación Orientada a Objetos 2026-1

Oscar Trujillo

Introducción

Este documento corresponde al Momento II del proyecto final de Tejo Cósmico. Su objetivo es presentar el diseño detallado de la capa lógica del videojuego, las vistas y dinámicas de cada nivel, los sprites previstos, el diagrama de clases con relaciones y contenedore y la arquitectura concreta del agente inteligente.

La capa de interfaz gráfica (clases QGraphicsScene, QGraphicsView, QMainWindow, etc.) se aborda solo de forma marginal ya que la guia pide centrarse en la capa lógica para esta entrega. El diseño completo que integra ambas capas se presentará en la entrega final.

Cambios respecto al Momento I

La idea general planteada en el Momento I se mantiene, tejo en un universo espacial con identidad colombiana, dos niveles diferenciados, tres físicas, dos rasgos de personalidad, agente inteligente en el Nivel 2. Sin embargo, durante el diseño detallado surgieron algunas precisiones y ajustes que vale la pena documentar:

- **Estructura del agente:** inicialmente se pensó como un solo objeto monolítico, pero en el diseño se separó en dos clases (AgenteIA y MemoriaAgente) para que el componente de aprendizaje sea reusable y testeable independientemente.
- **Inclusión de GestorRecursos:** se añadió esta clase utilitaria para centralizar carga de sprites y sonidos, facilitando el manejo de excepciones cuando un recurso no se encuentra.

Vistas e interacciones por nivel

Nivel 1, La Cancha Lunar

Descripción detallada de la vista

La pantalla del Nivel 1 está dividida así, en la parte superior, una franja de HUD que muestra puntaje, mechas restantes, tejos disponibles y el timer de 90 segundos. El cuerpo principal es la vista lateral de la cancha con el horizonte lunar como fondo (la Tierra visible al fondo a la derecha). El Mocho está fijo en el lado izquierdo sobre una plataforma de lanzamiento. La cancha de tejo se encuentra en el lado derecho, a una distancia que requiere calcular el tiro. Entre el lanzador y la cancha hay obstáculos, rocas flotantes que oscilan verticalmente.

La cámara hace un scroll horizontal limitado, cuando el jugador apunta hacia arriba para tiros largos, la cámara se ajusta levemente para seguir la trayectoria predicha, sin perder al Mocho de vista.

Interacciones

- **Apuntar y lanzar:** el jugador mueve el mouse para definir el ángulo de lanzamiento, mantiene presionado el clic izquierdo para cargar fuerza (la barra de fuerza se llena visualmente) suelta el clic para lanzar.
- **Recolectar chicha cósmica:** cada cierto tiempo aparece una botella de chicha en algún punto de la cancha. Si un tejo la golpea El Mocho activa el estado Guarapero por 10 segundos.

- **Esquivar el viento solar:** el viento aparece anunciado por una alerta visual unos segundos antes; el jugador debe decidir si lanzar antes, esperar, o lanzar compensando.
- **Mechazo:** cuando un tejo impacta una mecha, hay explosión animada, sonido y suma de puntos. Si tres mechazos consecutivos sin fallar se activa el Ojo de águila.

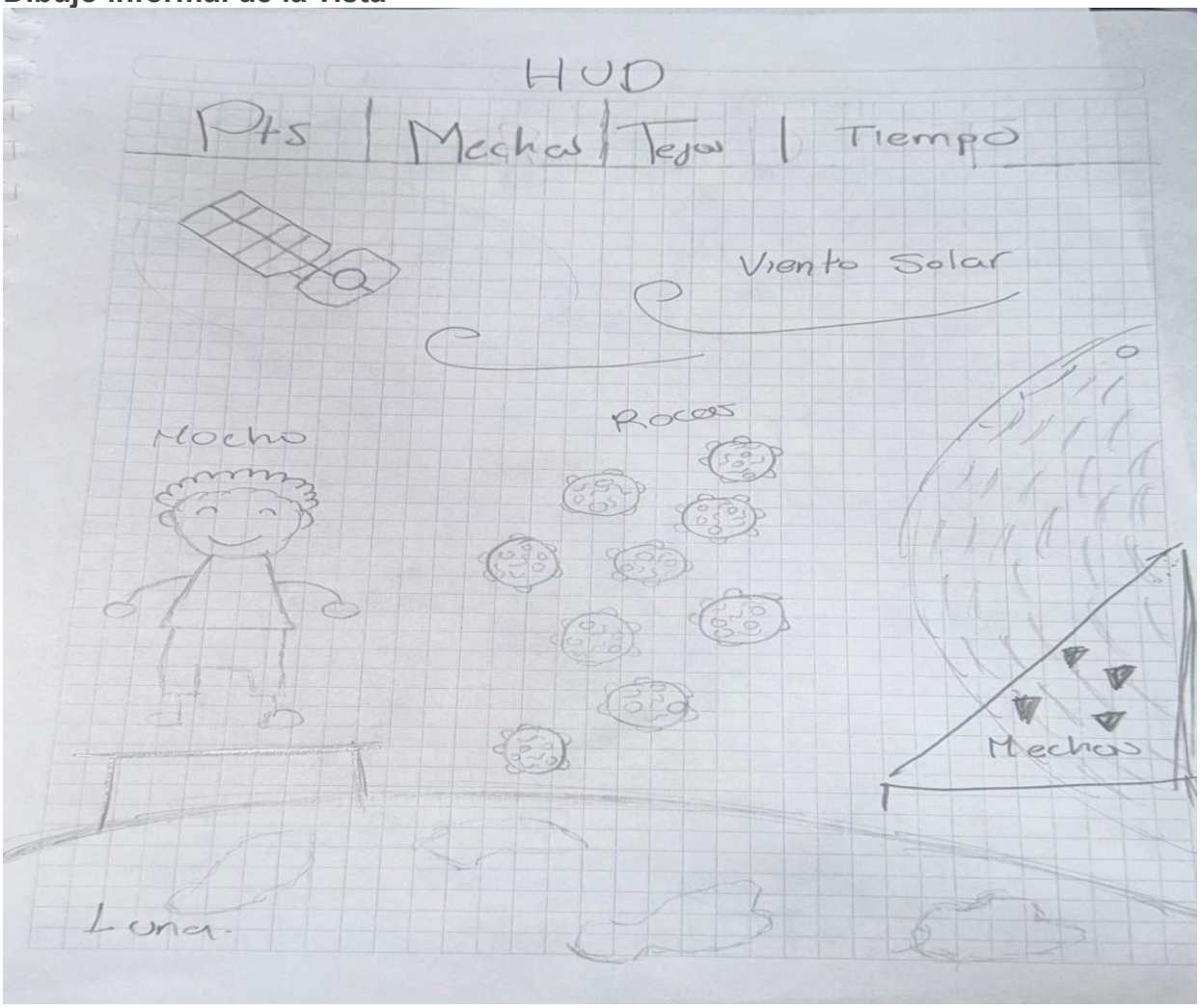
Físicas en este nivel (3)

1) Tiro parabólico con gravedad reducida. Ecuaciones paramétricas estándar de proyectil con $g = 1.62$ m/s² (gravedad lunar real). El jugador controla velocidad inicial v_0 y ángulo θ. Las posiciones se calculan como $x(t) = v_0 \cdot \cos(\theta) \cdot t$ y $y(t) = v_0 \cdot \sin(\theta) \cdot t - (1/2) \cdot g \cdot t^2$.

2) Movimiento oscilatorio de obstáculos. Las rocas flotantes oscilan verticalmente según $y(t) = y_0 + A \cdot \sin(\omega \cdot t + \varphi)$, con amplitud A entre 30 y 60 píxeles, frecuencia angular ω configurable por dificultad, y fase inicial φ aleatoria por roca.

3) Perturbación por viento solar. Durante el vuelo del tejo se aplica una fuerza horizontal F_viento en intervalos aleatorios. Esto modifica la aceleración horizontal del tejo en a_x = F_viento / m. La intensidad del viento es función de la dificultad seleccionada.

Dibujo informal de la vista



Nivel 2, El Anillo de Saturno

Descripción detallada de la vista

La pantalla muestra una plataforma circular flotante en el centro, con El Mocho en el medio. Alrededor de la plataforma se ven los anillos de Saturno como un fondo de partículas en movimiento lento. Los enemigos aparecen desde fuera del borde de la plataforma y se acercan al centro siguiendo trayectorias en zig zag. El HUD superior muestra puntaje, oleada actual, vidas (3 corazones) y tiempo total transcurrido.

A diferencia del Nivel 1, aquí el jugador puede apuntar en los 360 grados, no solo hacia un lado. La rotación del Mocho se sincroniza con la posición del mouse.

Interacciones

- **Apuntar en 360°:** el Mocho rota para encarar la posición del mouse, un clic lanza el tejo en esa dirección.
- **Rebote táctico:** lanzar un tejo contra los bordes magnéticos de la plataforma puede ser más eficiente para impactar enemigos en zonas difíciles.
- **Comandante Alien:** un enemigo especial aparece desde la oleada 2, tiene más vida que los demás y se mueve adaptándose al patrón del jugador.
- **Mecha en cadena:** si dos enemigos están cerca cuando uno explota, la explosión se propaga.

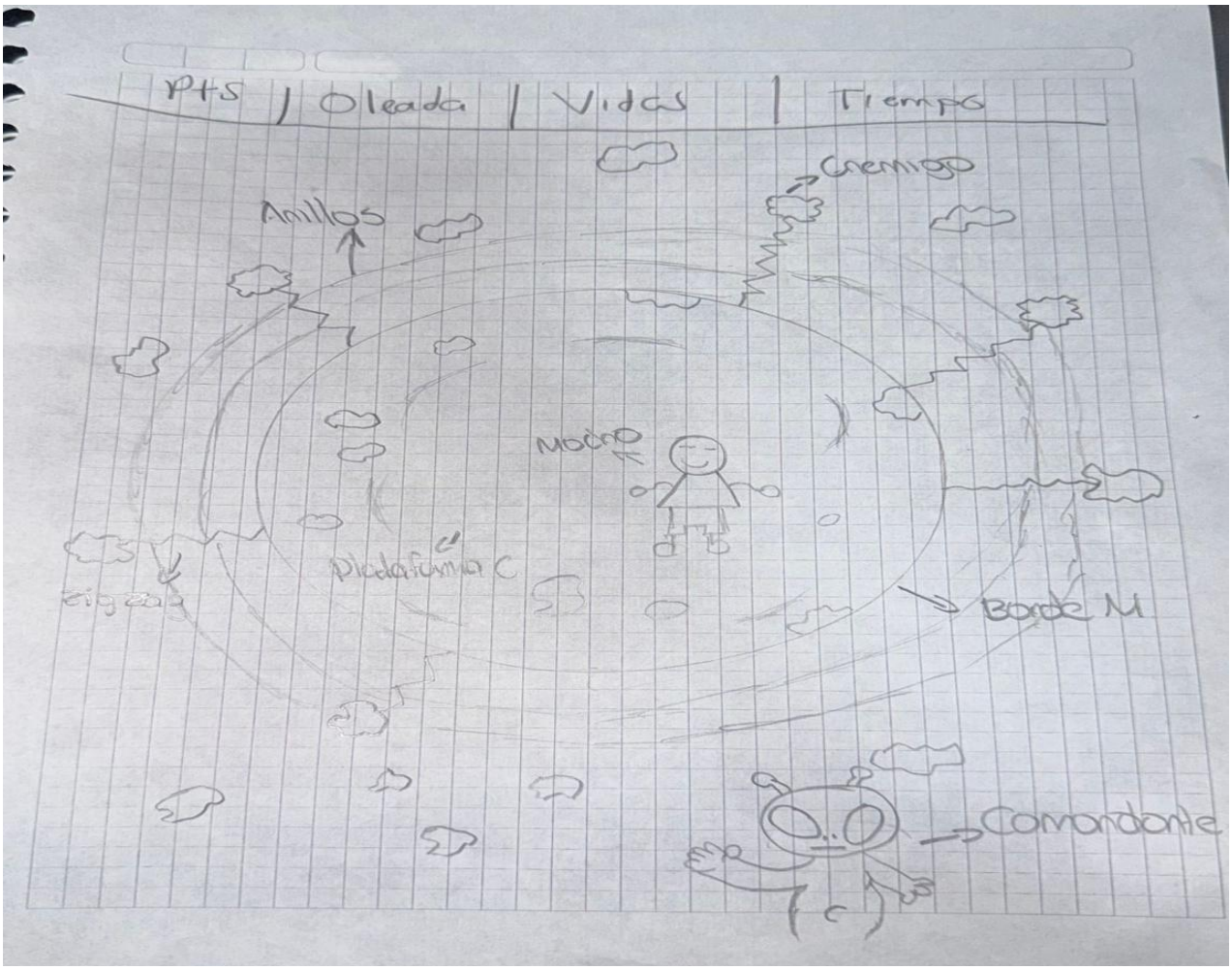
Físicas en este nivel (3)

1) Movimiento oscilatorio (senoidal en plano 2D). Los enemigos se desplazan hacia el centro siguiendo una trayectoria curva: $pos(t) = pos_inicial + v \cdot t + perpendicular \cdot A \cdot \sin(\omega \cdot t)$. La componente senoidal perpendicular a la dirección de avance crea el zig-zag.

2) Rebote elástico. Cuando un tejo impacta el borde magnético de la plataforma, se aplica reflexión vectorial: $v_nueva = v - 2 \cdot (v \cdot n) \cdot n$, donde n es la normal al borde. Se aplica un coeficiente de restitución $e < 1$ para que el tejo pierda algo de energía en cada rebote.

3) Atracción gravitacional residual. Saturno y sus anillos generan una pequeña fuerza atractiva $F = G \cdot M / r^2$ hacia el centro de la plataforma lo cual curva la trayectoria de los tejos. El jugador debe compensar apuntando ligeramente fuera del enemigo.

Dibujo informal de la vista



Sprites a utilizar (mínimo 4 por nivel)

A continuación se listan los sprites previstos para cada nivel. Los archivos correspondientes se anexan al directorio sprites del repositorio organizados por nivel. La fuente principal es OpenGameArt.org y bancos libres de itch.io.

Sprites del Nivel 1 (La Cancha Lunar)

Sprite	Descripción	Animaciones
mocholateral	Personaje principal en vista lateral, con su tejo. Animaciones de idle y lanzamiento.	Idle, lanzar
tejo	Disco metálico cilíndrico con brillo, vista lateral.	Rotación en vuelo
mecha	Triángulo de pólvora con mecha encendida, animación de chispa.	Chispeo, explosión
rocalunar	Roca espacial gris con cráteres, sombra inferior.	Estática (anim. por física)
fondolunar	Fondo: superficie lunar con la Tierra al horizonte.	Estático

Sprite	Descripción	Animaciones
chichacosmica	Botella estilo retro futurista con líquido brillante.	Brillo intermitente

Sprites del Nivel 2 (El Anillo de Saturno)

Sprite	Descripción	Animaciones
mochocenital	Mocho en vista cenital, con rotación circular completa.	8 direcciones, lanzar
enemigobasico	Criatura espacial pequeña con tentáculos.	Flotación, daño
comandantealien	Enemigo más grande, con casco metálico y aura.	Flotación, esquivar
plataformacircular	Plataforma metálica circular con anillo magnético en el borde.	Estático
fondosaturno	Anillos de Saturno con partículas y planeta visible.	Parallax lento
explosioncadena	Animación de explosión que se propaga.	Animada

Detalle de la herencia propia

El proyecto incluye varias jerarquías de herencia propia (no heredadas de Qt). La más relevante es la jerarquía Entidad → Personaje → Mocho.

Contenedores STL utilizados

- **std::vector<Entidad*> en Nivel:** almacena todas las entidades activas del nivel (tejos en vuelo, mechas, enemigos, obstáculos). Se eligió vector por acceso aleatorio rápido, recorrido eficiente para actualizar y dibujar, y porque la cantidad de elementos es moderada y no requiere inserciones intermedias.
- **std::unique_ptr<Nivel> en Juego:** el nivel actual se administra como puntero único. Garantiza que el nivel se destruya automáticamente al cambiar de nivel o cerrar el juego evitando fugas de memoria.
- **std::map<int,int> en MemoriaAgente:** mapea sector angular (clave) a número de veces que el jugador apuntó allí (valor). Map ofrece búsqueda y actualización O(log n) y mantiene un orden por clave útil para depuración.
- **std::vector<Fisica*> en Entidad:** cada entidad puede tener varios modelos físicos aplicados (por ejemplo, un tejo en el Nivel 1 sufre física parabólica + perturbación por viento). El vector permite iterar y acumular efectos.
-

Arquitectura del agente inteligente

El agente encarnado en la clase ComandanteAlien que delega su lógica en AgenteIA, está organizado en los cuatro componentes pedidos por la guia. A continuación se describe en alto nivel el flujo de cada componente.

Percepción

Disparador: cada vez que El Mocho lanza un tejo en el Nivel 2 el sistema emite un evento. El agente captura ese evento y registra dos datos: (a) el sector angular hacia el cual fue dirigido el tejo (la plataforma se divide en 8 sectores de 45 grados cada uno), y (b) si el lanzamiento impactó a algún enemigo o no.

Razonamiento

Cada vez que el agente debe decidir su próxima posición (típicamente una vez por segundo o al iniciar una nueva oleada), consulta a MemoriaAgente para obtener la distribución actual de sectores usados. La regla de decisión es: elegir el sector con el menor número acumulado de tiros, con un pequeño componente aleatorio para evitar volverse predecible.

Acción

Una vez decidido el sector, el agente actualiza los parámetros de su trayectoria oscilatoria (amplitud y frecuencia) para desplazarse hacia ese sector. La acción se ejecuta en cada llamada a actualizar(dt), donde la posición del Comandante se recalcula con base en la trayectoria objetivo.

Aprendizaje

El componente de aprendizaje vive en MemoriaAgente que mantiene el conteo acumulado entre oleadas (no se reinicia al cambiar de oleada). De manera opcional podría persistirse entre partidas leyendo y escribiendo a un archivo, pero para esta entrega se mantiene en memoria volátil. La clave del aprendizaje es que la distribución de probabilidades del jugador converge con el tiempo y el agente la aprovecha cada vez mejor.

Diagrama de flujo del agente (descripción)

El ciclo de vida del agente, descrito en prosa, es el siguiente: al inicio del Nivel 2 se crea el ComandanteAlien e instancia su AgenteIA con una MemoriaAgente vacía. En cada lanzamiento del jugador, la percepción se dispara y registra el sector y resultado. En cada actualización del juego (loop principal), el agente evalúa si toca decidir nuevo sector objetivo (cada N milisegundos o al detectar cambio de oleada) si toca razona consultando memoria, decide el sector menos usado por el jugador, y actúa ajustando trayectoria. La memoria se mantiene durante toda la partida, acumulando información, lo cual constituye el aprendizaje.